

LAB ASSIGNMENT – 13.5

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description #1 (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.

- o Refactor the code to pass values using function parameters.

- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1
```

```
def calculate_interest(amount):
```

```
    return amount * rate
```

```
print(calculate_interest(1000))
```

- Expected Output:

- o Refactored version passing rate as a parameter or using a configuration structure.

Code:

```
# Refactor the given Python code to remove the global variable and pass the value as a  
function parameter to improve modularity and testability
```

```
def calculate_interest(amount, rate):
```

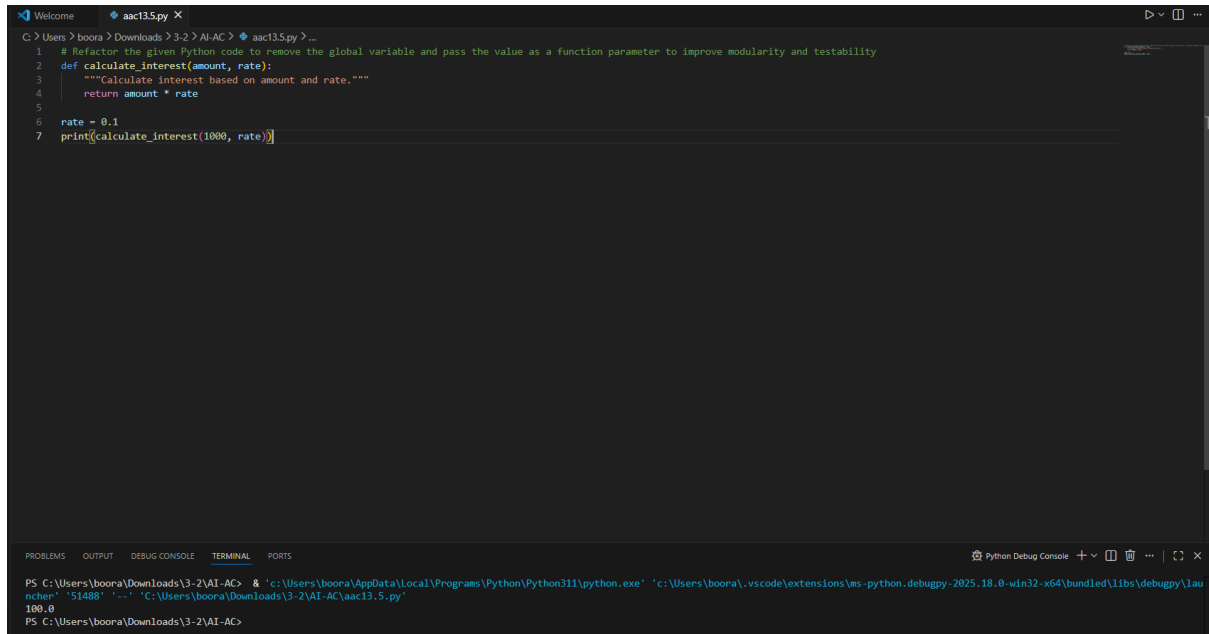
```
    """Calculate interest based on amount and rate."""
```

```
    return amount * rate
```

```
rate = 0.1
```

```
print(calculate_interest(1000, rate))
```

Output:



```
1 # Refactor the given Python code to remove the global variable and pass the value as a function parameter to improve modularity and testability
2 def calculate_interest(amount, rate):
3     """Calculate interest based on amount and rate."""
4     return amount * rate
5
6 rate = 0.1
7 print(calculate_interest(1000, rate))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
PS C:\Users\boora\Downloads\3-2\AI-AC> & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51488' '-.-' 'C:\Users\boora\Downloads\3-2\AI-AC\aac13.5.py'
100.0
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

- Focus Areas:

- o Readability

- o Logical simplification

- o Maintainability

Legacy Code:

```
score = 78
```

```
if score >= 90:
```

```
    print("Excellent")
```

```
else:
```

```
    if score >= 75:
```

```
print("Very Good")
```

```
else:
```

```
if score >= 60:
```

```
print("Good")
```

```
else:
```

```
print("Needs Improvement")
```

Expected Outcome:

- o Flattened logic using guard clauses or a mapping-based approach.

Code:

Refactor the given Python code to remove deeply nested if–else statements and simplify the logic for better readability and maintainability.

```
score = 78
```

```
if score >= 90:
```

```
    print("Excellent")
```

```
elif score >= 75:
```

```
    print("Very Good")
```

```
elif score >= 60:
```

```
    print("Good")
```

```
else:
```

```
    print("Needs Improvement")
```

Output:

```
9
10
11
12 # Refactor the given Python code to remove deeply nested if-else statements and simplify the logic for better readability and maintainability.
13 score = 78
14
15 if score >= 90:
16     print("Excellent")
17 elif score >= 75:
18     print("Very Good")
19 elif score >= 60:
20     print("Good")
21 else:
22     print("Needs Improvement")
```

Python Debug Console

```
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.8-win32-x64\bundle\libs\debugpy\launcher' '51344' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\aac13.5.py'
100.0
Very Good
```

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas:

- o DRY principle

- o Context managers

- o Function reuse

Legacy Code:

```
f = open("data1.txt")
```

```
print(f.read())
```

```
f.close()
```

```
f = open("data2.txt")
```

```
print(f.read())
```

```
f.close()
```

Expected Outcome:

- o Reusable function using with open() and parameters.

Code:

```
def read_file(filename):

    """Read and print contents of a file."""

    try:

        with open(filename, "r") as f:

            print(f.read())

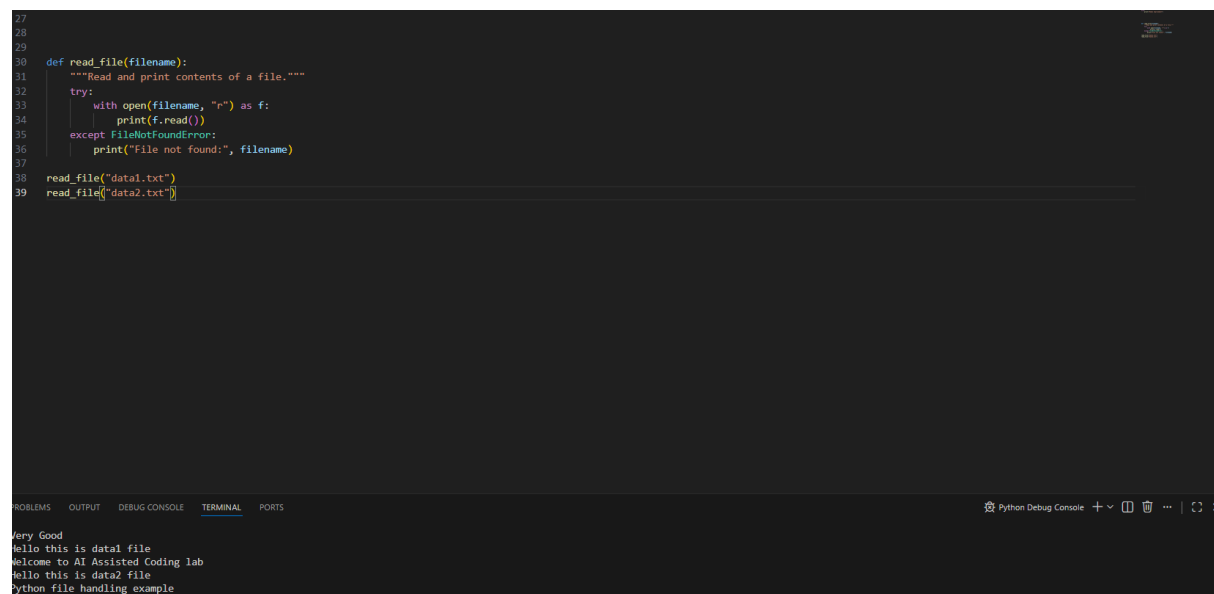
    except FileNotFoundError:

        print("File not found:", filename)

read_file("data1.txt")

read_file("data2.txt")
```

Output:

A screenshot of a Python IDE interface. The top pane shows a Python script with a function `read_file` that attempts to read and print the contents of a file. It includes a try-except block for `FileNotFoundError`. Below the function definition, the script calls `read_file("data1.txt")` and `read_file("data2.txt")`. The bottom pane shows the output of the execution: "Very Good", "Hello this is data1 file", "Welcome to AI Assisted Coding lab", "Hello this is data2 file", and "python file handling example". The interface includes a terminal window at the bottom with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active, showing the output text. There are also icons for Python Debug Console, a plus sign, a minus sign, a trash icon, and a refresh icon.

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]  
name = input("Enter username: ")  
  
found = False  
  
for u in users:  
    if u == name:  
        found = True  
  
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Code:

Refactor the given Python code to replace linear search with a set or dictionary for faster lookup and better time complexity.

```
users = {"admin", "guest", "editor", "viewer"}
```

```
name = input("Enter username: ")
```

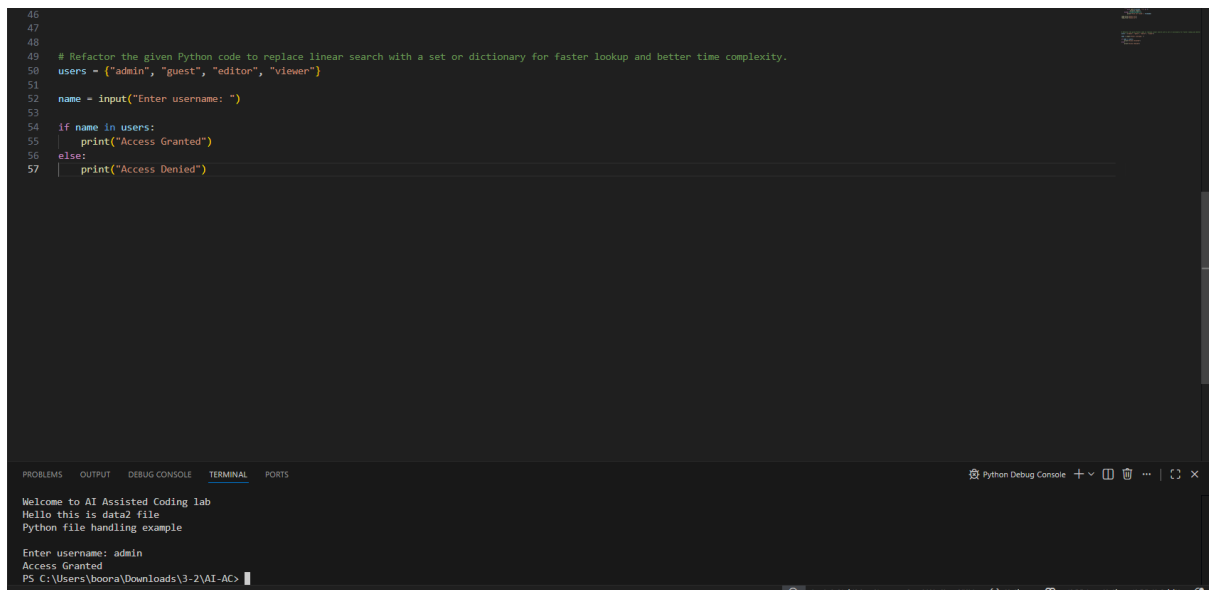
```
if name in users:
```

```
    print("Access Granted")
```

```
else:
```

```
    print("Access Denied")
```

Output:

A screenshot of a Python IDE interface. The main editor window shows a Python script with the following code:

```
46  
47  
48  
49 # Refactor the given Python code to replace linear search with a set or dictionary for faster lookup and better time complexity.  
50 users = {"admin", "guest", "editor", "viewer"}  
51  
52 name = input("Enter username: ")  
53  
54 if name in users:  
55     print("Access Granted")  
56 else:  
57     print("Access Denied")
```

The bottom panel of the IDE is split into two sections. The left section, labeled 'TERMINAL', shows the following output:

```
Welcome to AI Assisted Coding lab  
Hello this is data2 file  
Python file handling example  
  
Enter username: admin  
Access Granted  
PS C:\Users\boona\Downloads\3-2\AI-AC>
```

The right section of the bottom panel is labeled 'Python Debug Console' and is currently empty.

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

Code:

Refactor the procedural Python code into an object-oriented class design using encapsulation and methods.

```
class EmployeeSalaryCalculator:
```

```
    """Class to calculate employee net salary after tax."""
```

```
    def __init__(self, salary):
```

```
        self.salary = salary
```

```
        self.tax_rate = 0.2
```

```
    def calculate_tax(self):
```

```
        """Calculate tax amount."""
```

```
        return self.salary * self.tax_rate
```

```
    def calculate_net_salary(self):
```

```
        """Calculate net salary after tax."""
```

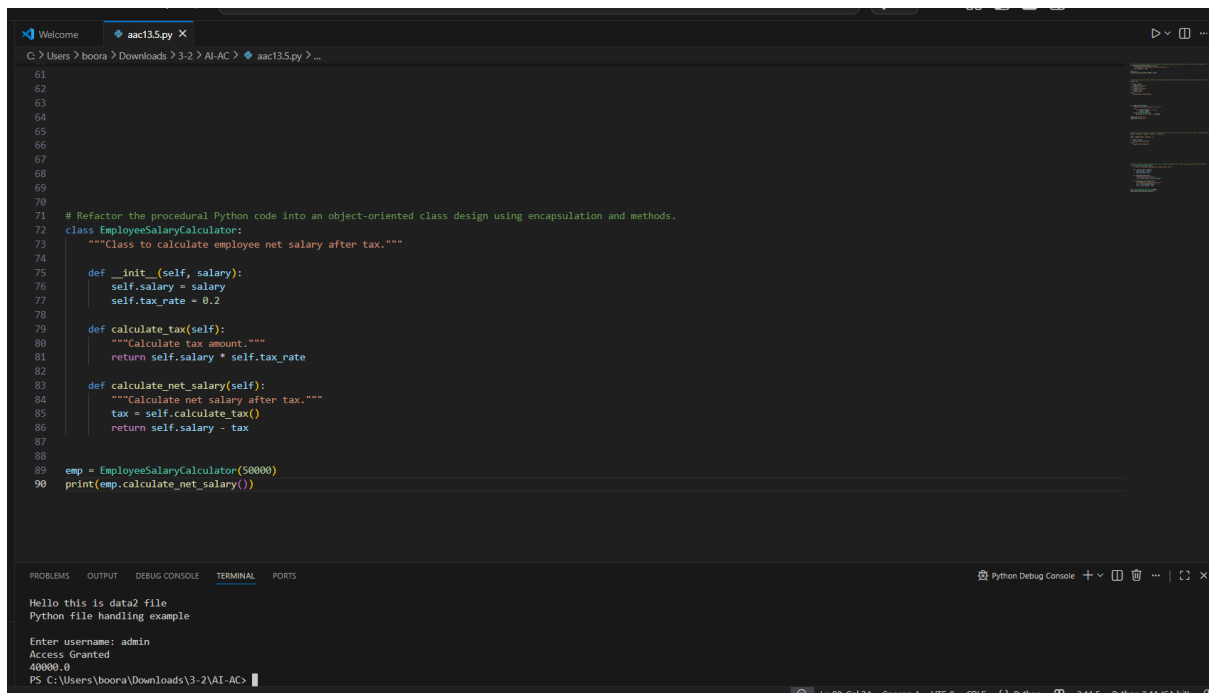
```
        tax = self.calculate_tax()
```

```
        return self.salary - tax
```

```
emp = EmployeeSalaryCalculator(50000)
```

```
print(emp.calculate_net_salary())
```


Output:



The screenshot shows a VS Code editor window with a file named 'aac13.5.py'. The code is a Python script that refactors procedural code into a class. The class is named 'EmployeeSalaryCalculator' and has two methods: 'calculate_tax' and 'calculate_net_salary'. The 'calculate_tax' method takes a salary and returns the tax amount. The 'calculate_net_salary' method takes a salary and returns the net salary after tax. The script creates an instance of the class with a salary of 50000 and prints the net salary.

```
61
62
63
64
65
66
67
68
69
70
71 # Refactor the procedural Python code into an object-oriented class design using encapsulation and methods.
72 class EmployeeSalaryCalculator:
73     """Class to calculate employee net salary after tax."""
74
75     def __init__(self, salary):
76         self.salary = salary
77         self.tax_rate = 0.2
78
79     def calculate_tax(self):
80         """Calculate tax amount."""
81         return self.salary * self.tax_rate
82
83     def calculate_net_salary(self):
84         """Calculate net salary after tax."""
85         tax = self.calculate_tax()
86         return self.salary - tax
87
88
89 emp = EmployeeSalaryCalculator(50000)
90 print(emp.calculate_net_salary())
```

The terminal output shows the following text:

```
Hello this is data2 file
Python file handling example

Enter username: admin
Access Granted
40000.0
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.
- Focus Areas:
 - o Algorithmic optimization
 - o Use of built-in functions

Legacy Code:

```
total = 0

for i in range(1, 1000000):

    if i % 2 == 0:

        total += i

print(total)
```

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

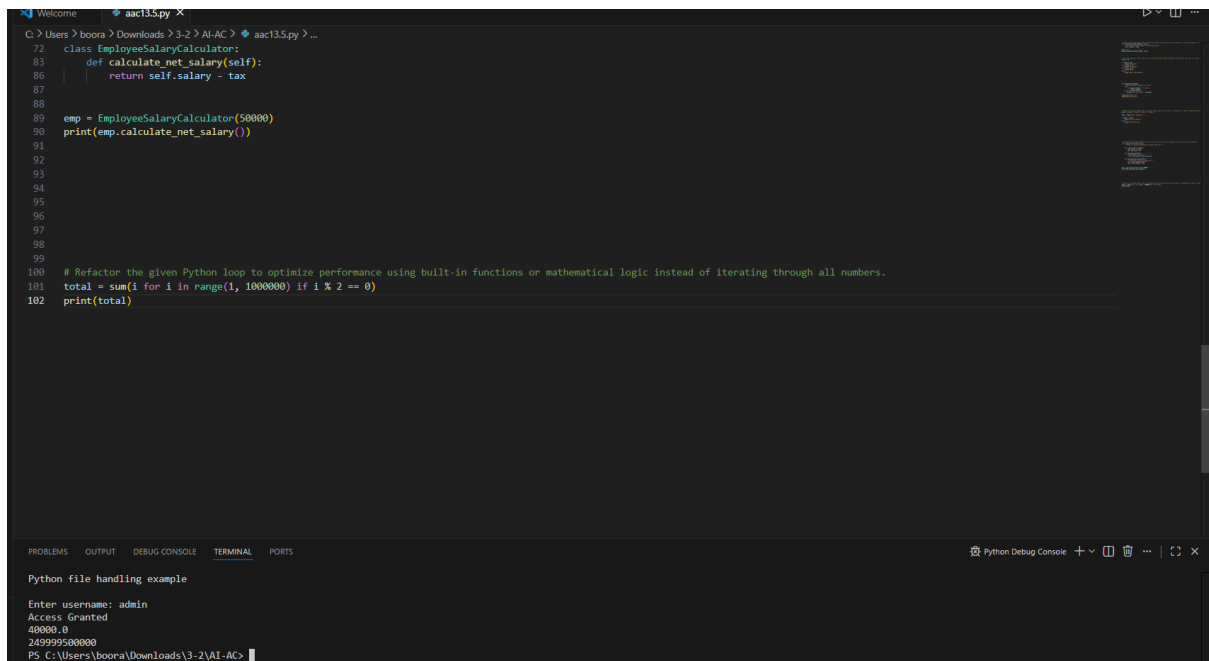
Code:

Refactor the given Python loop to optimize performance using built-in functions or mathematical logic instead of iterating through all numbers.

```
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
```

```
print(total)
```

Output:

A screenshot of a Python IDE window titled 'aac133.py'. The editor shows a class definition for 'EmployeeSalaryCalculator' with a method 'calculate_net_salary' that returns 'self.salary - tax'. Below this, an instance 'emp' is created with a salary of 50000, and its 'calculate_net_salary' method is called, printing 40000.0. A comment indicates the next task is to refactor a loop. The terminal at the bottom shows the output of the program: 'Enter username: admin', 'Access Granted', '40000.0', and '249999900000'. The terminal prompt is 'PS C:\Users\boora\Downloads\3-2\AI-AC>'.

```
72 class EmployeeSalaryCalculator:
73     def calculate_net_salary(self):
74         return self.salary - tax
75
76
77
78
79 emp = EmployeeSalaryCalculator(50000)
80 print(emp.calculate_net_salary())
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100 # Refactor the given Python loop to optimize performance using built-in functions or mathematical logic instead of iterating through all numbers.
101 total = sum(i for i in range(1, 1000000) if i % 2 == 0)
102 print(total)
```

Python file handling example

Enter username: admin
Access Granted
40000.0
249999900000
PS C:\Users\boora\Downloads\3-2\AI-AC>

Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.

- Focus Areas:

- o Functional-style refactoring

- o Predictability

Legacy Code:

```
data = []
```

```
def add_item(x):
```

```
    data.append(x)
```

```
add_item(10)
```

```
add_item(20)
```

```
print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Code:

Refactor the code to remove hidden side effects by avoiding modification of global variables and returning values from the function

```
def add_item(data, x):
```

```
    """Return a new list with the added item."""
```

```
    return data + [x]
```

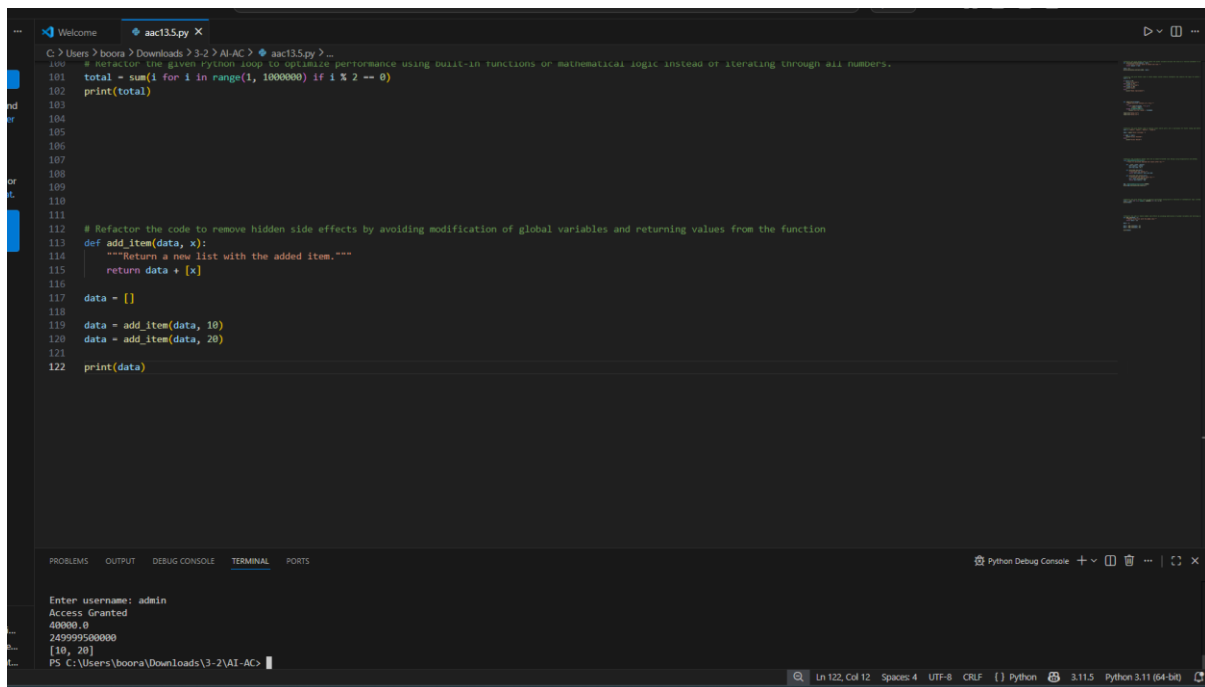
```
data = []
```

```
data = add_item(data, 10)
```

```
data = add_item(data, 20)
```

```
print(data)
```

Output:



The screenshot shows a VS Code editor with a file named 'aac135.py' open. The code in the editor is as follows:

```
100 # Refactor the given python loop to optimize performance using built-in functions or mathematical logic instead of iterating through all numbers.
101 total = sum(i for i in range(1, 1000000) if i % 2 == 0)
102 print(total)
103
104
105
106
107
108
109
110
111
112 # Refactor the code to remove hidden side effects by avoiding modification of global variables and returning values from the function
113 def add_item(data, x):
114     """Return a new list with the added item."""
115     return data + [x]
116
117 data = []
118
119 data = add_item(data, 10)
120 data = add_item(data, 20)
121
122 print(data)
```

The terminal at the bottom shows the output of the script:

```
Enter username: admin
Access Granted
4000000
249999500000
[10, 20]
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.

- Focus Areas:

- o Readability

- o Testability

Legacy Code:

```
password = input("Enter password: ")
```

```
if len(password) >= 8:
```

```
if any(c.isdigit() for c in password):
```

```
if any(c.isupper() for c in password):
```

```
print("Valid Password")
```

```
else:
```

```
print("Must contain uppercase")
```

```
else:
```

```
print("Must contain digit")
```

```
else:
```

```
print("Password too short")
```

Expected Outcome:

- o Separate validation functions with clear responsibility

Code:

Refactor the password validation code by separating validation rules into functions to improve readability and testability.

```
def is_length_valid(password):
```

```
    return len(password) >= 8
```

```
def has_digit(password):
```

```
    return any(c.isdigit() for c in password)
```

```
def has_uppercase(password):
```

```
    return any(c.isupper() for c in password)
```

```
password = input("Enter password: ")
```

```
if not is_length_valid(password):
```

```
    print("Password too short")
```

```
elif not has_digit(password):
```

```
    print("Must contain digit")
```

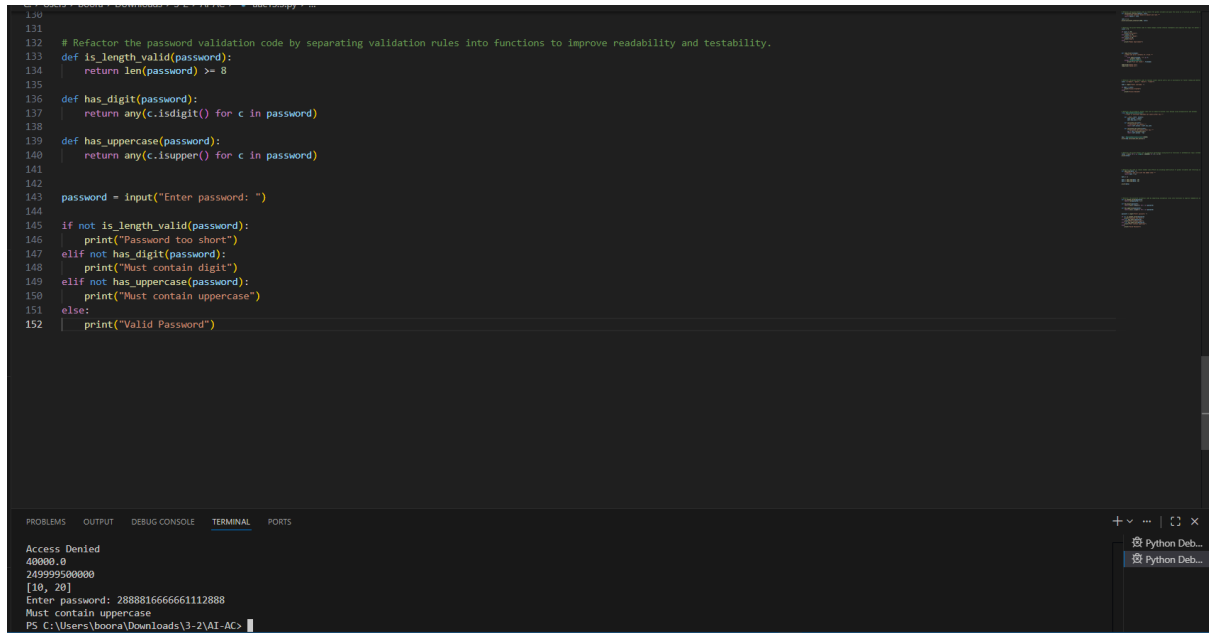
```
elif not has_uppercase(password):
```

```
    print("Must contain uppercase")
```

```
else:
```

```
print("Valid Password")'
```

Output:



The screenshot shows a code editor with a Python script for password validation. The code is as follows:

```
130
131
132 # Refactor the password validation code by separating validation rules into functions to improve readability and testability.
133 def is_length_valid(password):
134     return len(password) >= 8
135
136 def has_digit(password):
137     return any(c.isdigit() for c in password)
138
139 def has_uppercase(password):
140     return any(c.isupper() for c in password)
141
142
143 password = input("Enter password: ")
144
145 if not is_length_valid(password):
146     print("Password too short")
147 elif not has_digit(password):
148     print("Must contain digit")
149 elif not has_uppercase(password):
150     print("Must contain uppercase")
151 else:
152     print("Valid Password")
```

The terminal output at the bottom shows the execution of the script:

```
Access Denied
400000 0
2499999500000
[10, 20]
Enter password: 2888816666661112888
Must contain uppercase
PS C:\Users\boora\Downloads\3-2\AI-AC>
```